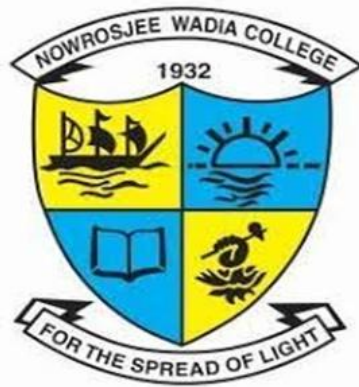




Modern Education Society's
Nowrosjee Wadia College

(An Autonomous college affiliated to Savitribai Phule Pune)

T.Y.B.Sc. (Computer Science) (NEP)

To be implemented from Academic Year 2025-2026

Semester – VI

WORKBOOK

Laboratory Course Based on Python Programming Language-II

Name: _____

Roll No.: _____ Division: _____

Academic Year: _____

Internal Examiner _____ External Examiner _____

INDEX

Sr. No.	Title of Practical	Remark (C / LC / IC)	Sign
1	Different Formats of Graph		
2	3D Plotting in Python		
3	Wireframes and Surface Plots of 3D Graphs		
4	Points and Operations on Points		
5	Lines and Linear Entities		
6	Polygon and Its Important Attributes		
7	Linear Programming Problem		
8	Transportation Problem		

CERTIFICATE

This is to certify that Mr./Ms. _____ Student of
T.Y.B.Sc. (Computer Science) Division – _____ Roll No. – _____
has successfully completed the Python Programming Language II course work during
the VI Semester of the Academic Year 2025–26 as per the details given below.

Practical Completion Table:

Total Number of Assignments	08
Number Of Assignments Completed (C)	
Number Of Assignments Completed Late (LC)	
Number Of Assignments Not Completed(IC)	

In-Charge
(Name & Sign)

Head of Department

Introduction

This laboratory workbook is designed as a practical companion to the Python Programming Language–II course for T.Y.B.Sc. (Computer Science) under the Choice Based Credit System (CBCS). This lab book aims to provide students with hands-on experience in applying Python programming to mathematical visualization, computational geometry, and operations research problems.

The experiments included in this file are structured according to the prescribed syllabus and focus on developing problem-solving skills through implementation, visualization, and analysis. The laboratory emphasizes the use of standard Python libraries such as NumPy, Matplotlib, SymPy, and PuLP, enabling students to translate mathematical concepts into efficient Python programs.

The syllabus covered in this lab book is divided into three major units, each targeting a specific application domain of Python.

Unit 1: 3D Graphs and Visualization

This unit introduces students to scientific and mathematical visualization using Python. It begins with the installation and usage of essential libraries such as NumPy and Matplotlib. Students learn to plot mathematical functions using different formats of graphs including line plots, bar charts, histograms, pie charts, scatter plots, contour plots, wireframe plots, and surface plots.

Special emphasis is given to three-dimensional points, lines, contour plots, wireframes, and surface plots, which help in understanding the behavior of functions of two variables. This unit strengthens the ability to visualize and interpret complex mathematical data graphically.

Unit 2: Computational Geometry

This unit focuses on the computational representation and manipulation of geometric objects using Python. Students study points, distance calculations, ordered point sets, extreme points, and random point sets. The unit further explores lines, rays, line segments, and polygons, including triangles and their properties.

Important geometric concepts such as collinearity, orientation tests (left, right, between), signed area of triangles, and relationships between points and lines are implemented programmatically. The unit also covers two-dimensional and three-dimensional transformations such as rotation and reflection, along with the generation of Bezier curves using given control points. This unit builds a strong foundation in geometry with applications in computer graphics and design.

Unit 3: Operations Research in Python

This unit introduces students to Operations Research techniques using Python. It includes the formulation and solution of Linear Programming Problems (LPP) and an introduction to the Simplex Method. Students learn how to model real-world optimization problems, define objective functions and constraints, and solve them using Python libraries such as PuLP. The unit also covers applied problems like transportation problems, helping students understand resource allocation, cost minimization, and decision-making through computational methods.

Instructions for Students:

1. Students must carry the lab book for every Python practical session.
No practical will be marked without the lab book.
2. All Python practicals must follow the department-prescribed format:
(Aim, Dataset/Input, Python Code, Output, Explanation/Conclusion).
3. Each Python practical must be completed before the next session and must be shown to the teacher for verification.
4. For Python-based practicals, students must:
 - Use only the allowed libraries/modules (e.g., NumPy, Pandas, Matplotlib, etc., as permitted by the department).
 - Clearly comment the code explaining each step.
5. Write Python code neatly with proper indentation, spacing, and meaningful variable names.
6. Maintain consistency in formatting:
All Python practicals must follow the same structure. Use proper margins and number all pages correctly.
7. Mobile phones are not allowed unless special permission is given by the teacher for reference purposes.
8. Students must get signatures for each completed practical on the designated signature page.
9. Late submission of Python practicals will not be accepted unless permitted by the teacher.
10. Copying Python code from classmates or the internet is strictly prohibited.
Students must write their own code, interpretation, and conclusion.
11. Ask doubts during the Python practical session.
Do not leave the lab without clearing your doubts.

Co-ordinator:

Dr. Poonam Ponde

Nowrosjee Wadia College, Pune

Prepared By:

Aditi Bhoite

Nowrosjee Wadia College, Pune

Assignment 1: Different Formats of Graphs

Aim: To study the basics of Python required for mathematical computations and data visualization, including installation of essential libraries and plotting various types of mathematical graphs using NumPy and Matplotlib.

Objectives:

- To install and understand the use of NumPy and Matplotlib packages.
- To perform basic numerical computations using NumPy.
- To plot different mathematical functions using Matplotlib.
- To visualize three-dimensional functions in two-dimensional form.
- To understand and plot different formats of graphs such as line, bar, histogram, pie, and scatter plots.

Reading / Theory:

Python provides powerful libraries such as **NumPy** and **Matplotlib** for numerical computation and graphical representation of mathematical functions.

1. Installation of NumPy and Matplotlib Packages

NumPy Installation

NumPy (Numerical Python) is a Python library used for numerical computations.
pip install numpy

Matplotlib Installation

Matplotlib is used for 2D and 3D plotting.

pip install matplotlib

The following are the important Python functions used in this assignment:

NumPy Functions:

1. **np.linspace(start, stop, step)**

Generates evenly spaced values within a given range. Commonly used for creating x-axis values.

2. **np.log(x)**

Computes the natural logarithm of x.

3. **np.sin(x), np.cos(x), np.tan(x)**

Used to calculate trigonometric function values.

4. **np.arcsin(x), np.arccos(x), np.arctan(x)**
Used to calculate inverse trigonometric function values.
5. **np.exp(x)**
Calculates the exponential value of x.
6. **np.meshgrid()**
Creates coordinate matrices from coordinate vectors, useful for plotting 3D functions in 2D form.

Matplotlib Functions:

1. **plt.plot(x, y)**
Used to draw line graphs for mathematical functions.
2. **plt.scatter(x, y)**
Used to draw scatter plots to show relationships between variables.
3. **plt.bar(categories, values)**
Used to draw bar graphs for categorical data.
4. **plt.hist(data)**
Used to draw histograms showing data distribution.
5. **plt.pie(data)**
Used to draw pie charts representing proportional data.
6. **plt.contourf(X, Y, Z)**
Used to draw filled contour plots for visualizing 3D functions in 2D.
7. **plt.colorbar()**
Adds a color scale reference to contour plots.
8. **plt.title(), plt.xlabel(), plt.ylabel()**
Used to add title and axis labels to the graph.
9. **plt.legend()**
Displays labels for plotted data.
10. **plt.grid(True)**
Adds grid lines for better readability.
11. **plt.show()**
Displays the final graph on the screen.

Reference Question:

Draw a pie chart showing the percentage distribution of time spent in a day.

Activity	Study	Sleep	Playing	Mobiles	Others
Percentage	30	35	15	10	10

Step 1: Import Matplotlib library

```
import matplotlib.pyplot as plt
```

Step 2: Define the data

```
activities = ['Study', 'Sleep', 'Playing', 'Mobile', 'Others']
```

```
percentage = [30, 35, 15, 10, 10]
```

Step 3: Plot the pie chart

```
plt.pie(percentage, labels=activities)
```

Step 4: Add title to the graph

```
plt.title("Daily Time Distribution")
```

Step 5: Display the pie chart

```
plt.show()
```

SET :

Q.1) Write a Python program to visualize the three-dimensional function

$z = x^2 + y^2$ in two dimensions using NumPy and Matplotlib.

Q.2) Write a Python program to visualize the three-dimensional function

$z = \sin(x) + \cos(y)$ in two dimensions using Python.

Q.3) Write a Python program to plot a line graph showing the marks of a student in 5 subjects:

Subjects	Maths	CS	Physics	Chemistry	English
Marks	78	88	72	69	85

Q.4) Plot two lines on the same graph showing sales of two products (Product A & Product B) for 6 months:

Months	Jan	Feb	Mar	Apr	May	Jun
A	50	60	55	70	65	80
B	30	40	45	50	60	75

Q.5) Write a Python program to draw a bar graph showing the number of students in 5 different classes:

Class	FY	SY	Ty	M.Sc.-I	M.Sc.-II
Students	120	110	95	80	70

Q.6) Create a bar graph for the average rainfall in 5 months:

Months	Jan	Feb	Mar	Apr	May
Rainfall (mm)	80	60	100	150	120

Q.7) Draw the histogram for the following data showing the marks obtained by students:

Marks	0 - 10	10 - 20	20 - 30	30 - 40	40 - 50	50 - 60
No. of Students	3	5	9	14	11	8

Q.8) Draw a histogram for the given data showing the distribution of weights(in kg):

Weight	40 - 45	45 - 50	50 - 55	55 - 60	60 - 65
No. of Persons	5	9	14	10	7

Q.9) Draw the pie chart for the following data:

Grain	Wheat	Bajra	Rice	Maize	Jowar	Others
% of population	23	25	22	15	11	4

Q.10) Plot a pie chart of the time spent on daily activities:

Activity	Sleep	Study	Mobile	Entertainment	Others
Hours	8	4	3	2	7

Q.11) Create a scatter plot showing height vs weight of 8 people:

Heights = [150, 155, 160, 165, 170, 175, 180, 185]

Weights = [45, 50, 55, 60, 65, 70, 75, 80]

Q.12) Plot a scatter diagram showing age (in years) vs salary (in thousands):

Age = [22, 25, 28, 30, 35, 40, 45, 50]

Salary = [20, 30, 35, 40, 50, 60, 70, 80]

Signature: _____

Date: _____

Assignment Evaluation:

1: Late Complete

☐

2: Incomplete

☐

3: Complete

☐

Assignment 2: 3D Plotting in Python

Aim: To study and implement three- dimensional (3D) plotting using matplotlib and numpy, and to visualize 3D points, lines and surface through basics 3D graph plotting functions.

Objectives:

1. To understand how to import and use libraries required for 3D plotting.
2. To learn the syntax and functions of matplotlib for creating 3D plots.
3. To plot 3D points and 3D line graphs using python.

Reading / Theory:

Three-dimensional (3D) plotting in Python is used to visualize data that depends on three variables. Unlike 2D plots that use only X and Y axes, 3D plots use X, Y, and Z axes to represent complex relationships in data. Python provides the **Matplotlib** library along with the **mpl_toolkits.mplot3d** module to create three-dimensional visualizations.

Three-Dimensional Points and Lines

In 3D plotting, each data point is represented as a coordinate triplet (x, y, z). These points can be connected or displayed individually to form different types of plots. To enable 3D plotting in Matplotlib, a 3D axis is created using the parameter **projection='3d'**. This allows plotting data in three-dimensional space.

➤ 3D Line Plot

A three-dimensional line plot represents a continuous path in space formed by joining a sequence of (x, y, z) points. It is commonly used to visualize curves, trajectories, or mathematical functions in three dimensions. The function **ax.plot3D()** is used to create 3D line plots.

➤ 3D Scatter Plot

A three-dimensional scatter plot displays individual data points in space without connecting them. Each point is plotted independently using (x, y, z) coordinates. This type of plot is useful for analyzing distributions, clusters, and spatial data. The function **ax.scatter3D()** is used for 3D scatter plotting.

Functions Used in 3D Contour Plot

1. import numpy as np

- Imports the **NumPy** library.
- NumPy is used for numerical computations and mathematical operations.
- It helps in creating arrays, performing vectorized calculations, and handling large datasets efficiently.

2. import matplotlib.pyplot as plt

- Imports the **Pyplot** module from the **Matplotlib** library.
- Provides functions to create figures, axes, and different types of plots.
- Used to display graphical output in Python.

3. `from mpl_toolkits import mplot3d`

- Imports the mplot3d toolkit.
- Enables three-dimensional plotting capabilities in Matplotlib.
- Required for creating 3D axes and visualizing 3D plots.

4. `fig = plt.figure()`

- Creates a new figure object.
- The figure acts as a container that holds all plot elements such as axes, titles, and labels.
- It is required before adding axes or plots.

5. `ax = plt.axes(projection='3d')`

- Creates a three-dimensional axes system.
- The argument `projection='3d'` enables plotting along X, Y, and Z axes.
- Allows visualization of three-dimensional data.

6. `ax.contour3D(x, y, z, n)`

- Creates a three-dimensional contour plot.
- `x`, `y`, and `z` represent data points in 3D space.
- `n` specifies the number of contour levels.
- Used to visualize surface variation using contour lines in three dimensions.

7. `ax.set_xlabel('x')`

- Sets the label for the X-axis.
- Improves clarity and readability of the graph.

8. `ax.set_ylabel('y')`

- Sets the label for the Y-axis.
- Helps in identifying the horizontal dimension.

9. `ax.set_zlabel('z')`

- Sets the label for the Z-axis.
- Helps in identifying the vertical dimension of the plot.

10. `plt.show()`

- Displays the final 3D contour plot.
- Executes all plotting commands and renders the graph on the screen.

Reference Question:

Generate 3D contour plots for the following function

$$f(x, y) = \sin(x + y), \text{ for } x \in [-2\pi, 2\pi], y \in [-2\pi, 2\pi]$$

Step 1: Import required libraries

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits import mplot3d
```

Step 2: Define the range for x and y

```
x = np.linspace(-2*np.pi, 2*np.pi, 100)
y = np.linspace(-2*np.pi, 2*np.pi, 100)
```

Step 3: Create a meshgrid

```
X, Y = np.meshgrid(x, y)
```

Step 4: Define the function $f(x, y) = \sin(x + y)$

```
Z = np.sin(X + Y)
```

Step 5: Create a figure and a 3D axis

```
fig = plt.figure(figsize=(8,6))
```

```
ax = plt.axes(projection='3d')
```

Step 6: Plot 3D contour

```
ax.contour3D(X, Y, Z, 50, cmap='viridis')
```

Step 7: Add axis labels and title

```
ax.set_xlabel('X-axis')
```

```
ax.set_ylabel('Y-axis')
```

```
ax.set_zlabel('Z-axis')
```

```
ax.set_title('3D Contour Plot of  $f(x,y) = \sin(x+y)$ ')
```

Step 8: Show the plot

```
plt.show()
```

SET

Q1. Generate 3D contour plots for the following function:

1. $f(x, y) = x^3 + y^2$ for $x \in [0, 2\pi]$, $y \in [0, 2\pi]$.
2. $f(x, y) = \sin(x^2 + y)$, for $x \in [-2\pi, 2\pi]$, $y \in [-2\pi, 2\pi]$.
3. $f(x, y) = \sin(x + y) + \cos x$, for $x \in [-2\pi, 2\pi]$, $y \in [-2\pi, 2\pi]$.
4. $f(x, y) = \sin x + \cos x$, for $x, y \in [-2\pi, 2\pi]$.
5. $f(x, y) = \sin(x^2 + y^2)$, for $x, y \in [-6, 6]$.
6. $f(x, y) = x^2 + y^2$ for $x, y \in [-6, 6]$.
7. $f(x, y) = e^{-x^2 - y^2}$ for $x, y \in (-1, 1)$.
8. $f(x, y) = \frac{\sin(4x) - \cos(5y)}{5}$ for $x, y \in (-1, 1)$.

Signature: _____

Date: _____

Assignment Evaluation:

1: Late Complete

☐

2: Incomplete

☐

3: Complete

☐

Assignment 3: Wireframes and Surface plots of 3D plots.

Aim: To understand and implement 3D Wireframe and 3D Surface plots using Python (Matplotlib), and analyze mathematical functions visually in three dimensions.

Objectives:

1. Understand the concept of 3D coordinate systems.
2. Plot 3D wireframe graphs using Matplotlib.
3. Plot 3D surface graphs using Matplotlib.
4. Develop Python code using NumPy and Matplotlib libraries.

Reading Theory:

Introduction

This assignment focuses on the visualization of three-dimensional mathematical functions using wireframe plots and surface plots in Python. These plots are used to represent functions of two variables in the form

$$z = f(x, y)$$

and help in understanding the shape, structure, and behavior of 3D surfaces.

Wireframe and surface plots are widely used in mathematics, engineering, and scientific analysis for better interpretation of complex data.

Libraries Used:

- NumPy
- Matplotlib (mplot3d toolkit)

❖ Wireframe Plot

A wireframe plot is a three-dimensional graphical representation of a surface created using a network of straight lines. It displays the structure of a surface by showing only the grid formed by the intersecting lines, without applying any color or shading.

Wireframe plots are useful for understanding the shape, curvature, and geometry of a three-dimensional function, as they allow visibility of all parts of the surface without obstruction.

Syntax of Wireframe Plot:

```
ax.plot_wireframe(X, Y, Z)
```

Here, X, Y, and Z are coordinate matrices obtained using `np.meshgrid()`.

❖ Surface Plot

A surface plot is a three-dimensional visualization in which the surface is drawn as a continuous colored sheet. The variation in colors across the surface represents changes in the values of the function. Surface plots provide a clearer and more realistic representation of three-dimensional data, making it easier to interpret the magnitude and behavior of the function over a given range.

Syntax of Surface Plot:

```
ax.plot_surface(X, Y, Z)
```

In this syntax, X, Y, and Z represent grid values of the function to be plotted.

Reference Question :

Generate 3D wireframes plots for the following functions:

$$f(x, y) = e^{x^2+y^2} \text{ for } x, y \in [0, 2\pi].$$

Step 1: Import required Python libraries

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits import mplot3d
```

Step 2: Define the range of x and y

```
x = np.linspace(0, 2*np.pi, 50)
y = np.linspace(0, 2*np.pi, 50)
```

Step 3: Create a meshgrid

```
X, Y = np.meshgrid(x, y)
```

Step 4: Define the function

```
Z = np.exp(X**2 + Y**2)
```

Step 5: Create a 3D figure

```
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
```

Step 6: Plot the wireframe

```
ax.plot_wireframe(X, Y, Z, color='blue', linewidth=0.7)
```

Step 7: Add labels and title

```
ax.set_title("3D Wireframe Plot of  $f(x,y) = e^{(x^2 + y^2)}$ ")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_zlabel("f(x, y)")
```

Step 8: Display the plot

```
plt.show()
```

SET :

Q1. Generate 3D wireframes plots for the following functions:

1. $f(x, y) = e^{x^2+y^2}$ for $x, y \in [0, 2\pi]$.
2. $f(x, y) = e^{x^2-y^2}$ for $x, y \in [0, 2]$.
3. $f(x, y) = e^{-x^2-y^2}$ for $x, y \in [0, 2\pi]$.
4. $f(x, y) = xe^{-x^2-y^2}$ for $x, y \in [-6, 6]$.
5. $f(x, y) = \sin x + \cos y$ for $x, y \in [-5, 5]$.

Q.2) Generate 3D surface plot's for the following functions.

1. $f(x, y) = xe^y$ for $x, y \in [0, 2]$.
2. $f(x, y) = x^3 + y^3 + x^2 + y^2 + x + y + 5$ for $x, y \in [-2, 2]$.
3. $f(x, y) = x^2 + y^2$ for $x, y \in [-6, 6]$.
4. $f(x, y) = \cos(|x| + |y|)$ for $x, y \in (-1, 1)$.
5. $f(x, y) = \cos(x^2 + y^2 - 0.5) - 0.5$ for $x, y \in (-1, 1)$.

Signature: _____

Date: _____

Assignment Evaluation:

1: Late Complete ☐

2: Incomplete ☐

3: Complete ☐

Assignment 4: Point's and operations on Point's.

Aim: To represent points in python and perform basic geometric operations such as finding distance, checking collinearity and coplanarity and applying transformation like scaling, reflection, shearing and rotation.

Objectives:

1. To understand the representation of 2D and 3D points in Python.
2. To calculate the distance between two points.
3. To check whether given points are collinear or coplanar.
4. To apply geometric transformations- scaling, reflection, shearing, and rotation on points using Python.

Reading:

a) Representation of Points

- A point in 2D is written as:

$$P(x, y)$$

- A point in 3D is written as:

$$P(x, y, z)$$

- In Python, two dimensional points are represented using **Point(x, y)** function

b) Distance Between Two Points

- **2D Distance Formula**

For Points $P(x_1, y_1)$ and $Q(x_2, y_2)$:

$$PQ = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- **3D Distance Formula**

For Points $P(x_1, y_1, z_1)$ and $Q(x_2, y_2, z_2)$:

$$PQ = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

- In Python, **distance()** function euclidean distance between two points is calculated.

c) Collinearity

- **Definition:** Three points A, B, and C in a plane are said to be **collinear** if they lie on the **same straight line**.
- In Python , we can check whether given three points are co- linear or not using **Point.is_collinear()**

d) Coplanarity

- **Definition:** A set of points is said to be **coplanar** if all the points lie **on the same plane**.
- In Python , we can check whether given three points are co- planar or not using **Point.is_coplanar()**

e) Transformations of Points

- **Scaling:** Scaling is a transformation that **changes the size** of a shape or object, either enlarging or shrinking it, while keeping its **shape the same**.
 - Scalling can be multiplying matrix $[T] = \begin{bmatrix} a & 0 \\ 0 & d \end{bmatrix}$ to point $X=(x,y)$
 - In Python, this can be evaluate using **X.scale(a, d)** function.
- **Reflection:** Reflection is a transformation that produces a **mirror image** of a shape across a specific line (in 2D) or plane (in 3D).
- **Shearing:** **Shearing** is a transformation that **slants** the shape in a particular direction.
- **Rotation:** **Rotation** is a transformation that turns a point **around the origin** (called the center of rotation) by a certain angle and direction (clockwise or counterclockwise).

❖ In Python, a geometric transformation can be applied to a point by using the **.transform()** method.

If a point is represented as **X = (x, y)**, and we have a corresponding transformation matrix, the transformation is performed using:

Syntax: X.transform(Matrix)

Reference Question:

Check whether given three point's are co-linear and co-planar or not and also find the distance between points Y, Z and X, Z

X[0, 0, 0], Y[2, 2, 2], Z[-1, -1, -1]

Step 1:Import required library

```
from sympy import*
```

Step 2: Defining the points

```
X = Point(0, 2, -1)
```

```
Y = Point(0, 2, -1)
```

```
Z = Point(0, 2, -1)
```

Step 3: Checking collinearity

```
result1 = Point.is_collinear(X, Y, Z)
print("Are the points collinear?", result1)
```

Step 4: Checking coplanarity

```
result1 = Point.is_coplanar(X, Y, Z)
print("Are the points coplanar?", result1)
```

Step 5: Finding Distance between two points

```
Y.distance(Z)
X.distance(Z)
```

SET :

Q.1) Check whether given three point's are co-linear or not.

- i) $A[2, 2, 2]$, $B[-1, -1, -1]$, $C[3, 4, -7]$.
- ii) $A[0, 0, 0]$, $B[-2, -2, -2]$, $C[-7, -7, -7]$.

Q.2) Check whether given three point's are co-planer or not.

- i) $A[2, 2, 2]$, $B[-1, -1, -1]$, $C[3, 4, -7]$.
- ii) $A[0, 0, 0]$, $B[-2, -2, -2]$, $C[-7, -7, -7]$.

Q.3) Find the distance between point's x, y ; y, w and x, z if $x = [0, 0]$,
 $y = [2, 2]$, $z = [-1, -1]$ and $w = [3, 4]$.

Q.4) Apply each of the following transformation on the point $P[-2, 3, 5]$.

- a. Reflection through the Y-axis.
- b. Scaling in X-coordinate by factor 2.
- c. Scaling in Y-coordinate by factor 1.5.
- d. Shearing in Y direction by 2 units.
- e. Scaling in X and Y direction by $\frac{1}{2}$ and 3 units respectively.
- f. Shearing in both X and Y direction by -2 and 4 units respectively.
- g. Rotation about origin by an angle 30 degree

Q.5) Apply each of the following transformations on the point $P[3, -1]$.

- a. Reflection through X-axis.
- b. Scaling in X-coordinate by factor 2.
- c. Scaling in Y-coordinate by factor 1.5.
- d. shearing in Y direction by 2 units.

- e. Scaling in X and Y direction by $\frac{1}{2}$ and 3 units respectively.
- f. Shearing in both X and Y direction by -2 and 4 units respectively.
- g. Rotation about origin by an angle 30 degree.

Signature: _____

Date: _____

Assignment Evaluation:

1: Late Complete ☐ **2: Incomplete** ☐ **3: Complete** ☐

Assignment 5: Lines and Linear Entities

Aim: To study and represent points, lines, rays, and line segments, and to understand their properties and geometric characteristics.

Objectives:

- To understand the differences between lines, rays, and line segments.
- To observe the geometric properties of line segments, including their length and endpoints.
- To develop the ability to represent linear entities accurately using diagrams and notation.
- To apply geometric transformations: rotation, reflection.
- To calculate distance, length, slope and midpoint.

Reading:

- ❖ **Line:** A line is a one-dimensional figure, which has length but no width. A line is made of a set of points which is extended in opposite directions infinitely.

Different forms:

- Slope-intercept: $y = mx + c$
- Two –point form:
$$\frac{y-y_1}{y_2-y_1} = \frac{x-x_1}{x_2-x_1}$$
- General form: $ax + by + c = 0$

e.g. :

- In python, line L passing through points (2, 3) and (4, 3) is defined as follows:

```
L=Line(Point(2, 3), Point(4, 3))
```

- Line having equation $2x + 3y = 4$

```
L=Line(2 * x + 3 * y - 4)
```

```
L
```

```
>>> Line2D(Point2D(0, 4/3), Point2D(1, 2/3))
```

- Line with point (1, 4) and slope is 1

```
L=Line(Point(1, 4), slope=1)
```

```
L
```

```
>>> Line2D(Point2D(1, 4), Point2D(2, 5))
```

- We can display its equation and its coefficients as follows:

```
L.equation()
```

```
>>> -x + y - 3
```

```
L.coefficients
```

```
>>> (-1, 1, -3)
```

❖ **Line segment:** The line segment can be specified by its end-points.

- E.g. line segment s having end points $(0, 0)$ and $(0, 13)$ is defined as follows:

```
s=Segment((0, 0), (0, 13))
s
```

❖ **Ray :** A ray is a semi-line in the space with a source point and a direction. It can be specified by two points, one of is source point.

- E.g. line segment s having end points $(0, 0)$ and $(3, 1)$ is defined as follows:

```
r=Ray((0, 0), (3, 1))
r
```

❖ **Functions related to lines, rays and segments:**

1. Angle between two linear entities(lines, rays, segments)

We can find angle formed by intersection of linear entities using **.angle_between()**

2. Intersection points of two linear entities:

We can find intersection point formed by intersection of linear entities using **.intersection()**

3. Length linear entities:

We can find length of linear entities using **.length()**

4. Slope of linear entities:

We can find the slope of the any linear entities using **.slope**

5. Midpoint of segment:

We can find the midpoint of segment using **.midpoint**

6. Points of linear entities:

Two points of any linear entities can be display using function **.points**

7. Rotation of linear entities:

Counter clock wise rotation of the any linear entities can be done using function

.rotation(angle) , where angle should be in radians.

8. Reflection of linear entities:

Reflection through a line of the any linear entities can be done using function **.reflcetion(line)**

Reference Question:

Reflect the segment having two endpoints $(5, 3)$ & $(1, 4)$ through the line $y = x + 1$

Step 1: Import SymPy library

```
from sympy import*
```

Step 2: Declare symbolic variables

```
x, y = symbols('x, y')
```

Step 3: Create the two endpoints of the segment

```
A = Point(5, 3)
```

```
B = Point(1, 4)
```

Step 4: Create the segment using the two points

```
S = Segment(A, B)
```

Step 5: Reflect the segment through the line

S.reflect(Line($x-y+1$))

SET :

Q1) Rotate the line by 30 degrees having two points (0,0) and (0,1). Also find it's equation after applying rotation.

Q2) Rotate the segment by 180 degrees having end points (1,0) and (2, -1).

Q3) Rotate the ray by 90 degrees clockwise & anticlockwise having starting point (0,0) in the direction of (4,4).

Q4) Reflect the line $4x + 3y = 5$ through the line $x + y = 0$ and find the equation of reflected line.

Q5) Reflect the segment having two endpoints (2,3) & (4,6) through the line $7x + 6y = 3$.

Q6) Reflect the ray having starting point (0,0) in the direction of (2,4) through the line $x - 2y = 3$.

Q7) If the line with points $A [2 \ 1], B [4 \ -1]$ is transformed by the transformation matrix, $[T] = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$, then find the equation of transformed line.

Q8) If the line segment with endpoints $A[11], B[-4 - 1]$ is transformed by the transformation matrix, $[T] = \begin{bmatrix} 1 & -2 \\ -2 & 1 \end{bmatrix}$, then find the end points of the transformed line.

Q9) If the line segment joining the points $A[25], B[4 - 13]$ is transformed to the line segment A^*B^* by the transformation matrix, $[T] = \begin{bmatrix} 2 & 3 \\ 4 & 1 \end{bmatrix}$, then find the midpoint of A^*B^* & also find slope.

Q10) If we apply shearing on the line $2x + y = 3$ in x & y direction by 2 & -3 units respectively, then find the equation of the resulting line.

Signature: _____

Date: _____

Assignment Evaluation:

1: Late Complete

☐

2: Incomplete

☐

3: Complete

☐

Assignment 6: Polygon & It's Important Attributes

Aim: To study polygons, their representation in Python, and to understand their important geometric attributes and transformations.

Objectives:

- To learn how to represent polygons in Python.
- To understand key attributes of polygons such as area, angles, convexity, perimeter, and center.
- To apply geometric transformations including reflection, rotation, and scaling.
- To study and identify types of triangles: isosceles, right-angled, and scalene.

Reading:

- ❖ **Polygon:** A polygon is a closed two-dimensional figure formed by connecting a finite number of line segments.

In Python , a polygon is created using:

from sympy import *

P = Polygon(Point(x1, y1), Point(x2, y2), ...)

- E.g. consider the polygon with the vertices (0, 0), (1, 0), (5, 1), (0, 1), (3, 0)

```
>>>p1, p2, p3, p4, p5 = [(0, 0), (1, 0), (5, 1), (0, 1), (3, 0)]  
>>>P=Polygon(p1, p2, p3, p4, p5)  
>>>P
```

- ❖ **Regular Polygon:**

A regular polygon is a polygon in which all sides are equal in length and all interior angles are equal. A regular polygon can be constructed by specifying three parameters: **centre**, **radius**, and **number of sides(n)**

Function: RegularPolygon(center, radius, n)

- ❖ **Important attributes of Polygon:**

- **Area:**
The amount of 2D space enclosed by the polygon. To calculate area we use **P.area**
- **Angles at Vertices:** The internal angles at each vertex of the polygon can be calculate using **P.angles(point)**
- **Convex Polygon:** A polygon is convex if all interior angles are less than 180° and are no intersection between sides. We can check whether given polygon is convex or not using **is_convex()**
- **Perimeter of Polygon:** The total length of the boundary of the polygon.
To calculate perimeter we use **P.perimeter**

- **Center of a Regular Polygon:** To calculate perimeter we use **P.center**
- **Scale :** For scaling polygon we use **P.scale(point)**

❖ Triangles:

Polygon with three vertices or three sides is a triangle.

```
e.g. >>> p1, p2, p3=[(0, 0), (1, 0), (5, 1)]
>>> P=Triangle(p1, p2, p3)
>>> P
```

Keywords sss, sas or asa can be used to give the desired side lengths(in order) and interior angles(in degrees) that define the triangle(first vertex by default (0,0)) :

```
>>> #triangle with sides 3, 4, 5
>>> Triangle(sss=(3, 4, 5))
>>> #triangle with angle=30, side=1, angle=30
>>> Triangle(asa=(30, 1, 30))
>>> #triangle with side=1, angle=45, side=2
>>> Triangle(sas=(1, 45, 2))
```

1. Isosceles triangle:

An **isosceles triangle** is a triangle in which **two sides are equal in length**. The angles opposite the equal sides are also equal.

To check triangle is an isosceles triangle we use **triangle.is_isosceles()**

2. Right angle triangle:

A **right-angled triangle** is a triangle in which **one interior angle is exactly 90°**. The side opposite the right angle is called the **hypotenuse**, and it is the longest side of the triangle.

To check triangle is Right angle triangle we use **triangle.is_right()**

3. Scalene triangle:

A scalene triangle is a triangle in which all three sides are of different lengths.

To check triangle is Scalene triangle we use **triangle.is_scalene()**

Reference Question:

Find the area and the perimeter of the triangle ABC , where $A[0\ 0]$, $B[5\ 0]$, $C[3\ 3]$.

Step 1: Import SymPy library

```
from sympy import*
```

Step 2: Declare symbolic variables

```
x, y = symbols('x, y')
```


Step 3: Create the two endpoints of the segment

A = Point(0, 0)

B = Point(5, 0)

C = Point(3, 3)

Step 4: Create the triangle using the points

T = Triangle(A, B, C)

Step 5: Find the area and perimeter of the triangle

T.area

T.perimeter

SET :

Q1) Draw a polygon with vertices (0,0), (1,0), (2,2), (1,4) and find its area & perimeter.

Q2) Draw a regular polygon with 8 sides & radius 5 centred at origin and find its area & perimeter.

Q3) Draw a regular polygon with 7 sides and radius 1.5 centered at (2,2) & reflect it through line $x - y = 5$.

Q4) Draw a polygon with vertices (0,0), (2,0), (2,3), (1,6) and rotate by 180 degrees.

Q5) Reflect the polygon ABC through the line $y = 3$, where

$A[1\ 0], B[2\ -1], C[-1\ 3]$.

Q6) Find the angle at each vertices of the triangle ABC , where $A[0\ 0], B[2\ 2], C[0\ 2]$.

Q7) Check the following polygons (triangles) are Isosceles, Right angled and Scalene triangle or not

i) $A[0\ 0], B[4\ 0], C[2\ 4]$

ii) $A[0\ 0], B[4\ 0], C[4\ 3]$

iii) $A[0\ 0], B[4\ 0], C[1\ 4]$

Q8) Find the points of triangle where we have sides and angles

i) $sss = (3,4,5)$

ii) $asa = (30,1,30)$

iii) $sas = (1,45,2)$

Signature: _____

Date: _____

Assignment Evaluation:

1: Late Complete

☐

2: Incomplete

☐

3: Complete

☐

Assignment 7: Linear Programming Problem

Aim: To study Linear Programming Problems (LPP), understand their mathematical formulation, and solve them using Python.

Objectives:

- To understand the concept of Linear Programming Problems.
- To learn the components of an LPP such as decision variables, objective function, and constraints.
- To formulate real-life problems as Linear Programming Problems.
- To solve Linear Programming Problems using Python.

Reading:

❖ Linear Programming Problem (LPP):

A Linear Programming Problem is a mathematical technique used to optimize (maximize or minimize) a linear objective function subject to a set of linear constraints. Linear Programming is widely used in areas such as business planning, production management, transportation, and resource allocation.

Python provides a rich ecosystem of comprehensive and powerful tools for solving Linear Programming Problems. One of the most popular Python packages for this purpose is PuLP. **PuLP** is an open-source linear programming library that allows users to define optimization problems in a simple and intuitive way using Python syntax.

In PuLP, the optimization problem is first modeled mathematically by defining decision variables, the objective function, and constraints. After modeling, the built-in PuLP solver module is called to compute the optimal values of the decision variables.

❖ General Mathematical Form of LPP:

Maximize / Minimize

$$Z = ax + by$$

Subject to:

- $a_1x + b_1y \leq c_1$
- $a_2x + b_2y \leq c_2$
- $x \geq 0, y \geq 0$

❖ Functions Used for Solving LPP

1. **LpProblem** : This function creates a new LP Problem with the specified associated parameters.
2. **LpVariable** : It is used to assign an LP Variable with the specified associated parameters.
3. **solve()** : It returns an integer status of the solution, which indicates whether the solution is optimal, infeasible, or unbounded. Consider the following table :

Solution Status	Numeric Output
Optimal	1
Not Solved	0
Infeasible	-1
Unbounded	-2
Undefined	-3

4. **objective.value:** It is used to display optimum value of objective function
5. **value:** It is used to display optimum value of associated variable.

Reference Question:

Solve the following LPP using python & find optimal solution if exist.

- a) $Max\ z = 3x + 2y + 5z$
 Subject to, $x + 2y + z \leq 430$
 $3x + 2z \leq 460$
 $x + 4y \leq 120$
 $x, y, z \geq 0$

Step 1: First import pulp

```
import pulp
from pulp import*
```

Step 2: Create model using LpProblem

```
model = LpProblem(sense = LpMaximize)
```

Step 3: Define the decision variables using the LpVariable

```
x = LpVariable(name = "x", lowBound=0)
y = LpVariable(name = "y", lowBound=0)
z = LpVariable(name = "z", lowBound=0)
```

Step 4: Add constraints to the model

```
model += (x+2*y+z <= 430)
model += (3*x+2*z <= 460)
model +=(x+4*y <= 120)
```

Step 5: Define the objective function

```
model += (3*x+2*y+5*z)
```

Step 6: Display the model

```
model
```

Step 7: Check the solution status

```
model.solve()
```

Step 8: Obtain the optimal value of the objective function

```
model.objective.value()
```

Step 9: Obtain the optimal value of decision variables

x.value()

y.value()

z.value()

SET :

Q. Solve the following LPP using python & find optimal solution if exist.

a) $Max\ z = 150x + 75y$

Subject to, $4x + 6y \leq 24$

$$5x + 3y \leq 15$$

$$x \geq 0, y \geq 0$$

b) $Min\ z = 3.5x + 2y$

Subject to, $x + y \geq 5$

$$x \geq 4$$

$$y \leq 2$$

$$x, y \geq 0$$

c) $Max\ z = 3x + 5y + 4z$

Subject to, $2x + 3y \leq 8$

$$2y + 5z \leq 10$$

$$3x + 2y + 4z \leq 15$$

$$x, y, z \geq 0$$

d) $Min\ z = x + 2y + z$

Subject to, $x + \frac{1}{2}y + \frac{1}{2}z \leq 1$

$$\frac{3}{2}x + 2y + z \geq 8$$

$$x, y, z \geq 0$$

e) $Min\ z = x + y$

Subject to, $x \geq 6$

$$y \geq 6$$

$$x + y \leq 11$$

$$x, y \geq 0$$

f) $Max\ z = x + y$

Subject to, $x - y \geq 1$

$$x + y \geq 2$$

$$x, y \geq 0$$

g) $Max\ z = 4x + y + 3z + 5w$
Subject to, $4x + 6y - 5z - 4w \geq -20$
 $-3x - 2y + 4z + w \leq 10$
 $-8x - 3y + 3z + 2w \leq 20$
 $x, y, z, w \geq 0$

Signature: _____

Date: _____

Assignment Evaluation:

1: Late Complete

☐

2: Incomplete

☐

3: Complete

☐

Assignment 8: Transportation Problem

Aim : To study the Transportation Problem as a special type of Linear Programming Problem and to understand its formulation and solution methods.

Objectives:

- To understand the concept of the Transportation Problem.
- To learn the mathematical formulation of the Transportation Problem.
- To study different methods used to obtain an initial feasible solution.
- To understand the optimality test for Transportation Problems.

Reading:

❖ **Transportation Problem:**

The Transportation Problem is a special type of Linear Programming Problem in which goods are transported from multiple supply points (sources) to multiple demand points (destinations) at minimum transportation cost. The main objective is to determine the optimal transportation schedule that satisfies supply and demand constraints while minimizing the total cost.

❖ **Types of Transportation Problems:**

1. **Balanced Transportation Problem:** When total supply is equal to total demand.
2. **Unbalanced Transportation Problem:** When total supply is not equal to total demand. A dummy source or destination is added to balance the problem.

❖ **Python Functions Used:**

1. **LpProblem :** Used to define the transportation problem as a minimization problem.
2. **LpVariable :** Used to define transportation variables representing quantity shipped from source i to destination j .
3. **lpSum() :** Used to define objective function and constraints as summation of linear expressions.
4. **solve() :** Solves the transportation problem and returns solution status.
5. **value() :** Used to extract optimal values of decision variables and total cost.

Reference Questions:

A boutique brewery has two warehouses from which it distributes beer to five carefully chosen bars. At the start of every week, each bar sends an order to the brewery's head office for so many crates of beer, which is then dispatched from the appropriate warehouse to the bar. The brewery would like to have an interactive computer program which they can run week by week to tell them which warehouse should supply which bar so as to minimize the costs of the whole operation. For example, suppose that at the start of a given week the brewery has 1000 cases at warehouse A and 4000 cases at warehouse B, and that the bars required 500, 900, 1800, 200 and 700 cases respectively. Transformation costs (dollars per crate) is given in the following table.

From warehouse to Bar	A	B
1	2	3
2	4	1
3	5	3
4	2	2
5	1	3

Which warehouse should supply which bar?

❖ **First formulate the given problem :**

1. Identify the Decision Variables:

A1= number of crates of beer to ship from warehouse A to Bar 1

A5= number of crates of beer to ship from warehouse A to Bar 5

B1= number of crates of beer to ship from warehouse B to Bar 1

B5= number of crates of beer to ship from warehouse B to Bar 5

Let $x_{(w, b)}$ be the variable assigned to rout from warehouse w to bar b. Then

$$W = \{A, B\}$$

$$B = \{1, 2, 3, 4, 5\}$$

$$x_{(w,b)} \in \mathbb{N} \cup \{0\}$$

2. Formulate the Objective Function:

If $c_{(w, b)}$ denotes the cost required to transport beer from warehouse w to bar b, then objective function will be

$$\text{Minimise the Transportation Costs} = \min \sum_{w \in W, b \in B} c_{(w,b)} x_{(w,b)}$$

3. Formulate the Constraints:

$$A1 + A2 + A3 + A4 + A5 \leq 1000$$

$$B1 + B2 + B3 + B4 + B5 \leq 4000$$

and

$$A1 + B1 \geq 500$$

$$A2 + B2 \geq 900$$

$$A3 + B3 \geq 1800$$

$$A4 + B4 \geq 200$$

$$A5 + B5 \geq 700$$

Python implantation:

Step 1: Import pulp module

```
import pulp
from pulp import*
```

Step 2: Creates a list of all the supply nodes

```
Warehouses = ["A", "B"]
```

Step 3: Creates a dictionary for the number of units of supply for each supply node

```
Supply = {"A":1000, "B": 4000 }
```

Step 4: Creates a list of all demand nodes

```
Bars = ["1", "2", "3", "4", "5"]
```

Step 5: Creates a dictionary for the number of units of demand for each demand node

```
Demand={"1":500, "2":900, "3": 1800, "4": 200, "5": 700}
```

Step 6: Creates a list of costs of each transportation path

```
Costs = {'A': {'1': 2, '2': 4, '3': 5, '4': 2, '5': 1},  
        'B': {'1': 3, '2': 1, '3': 3, '4': 2, '5': 3}}
```

Step 7: Creates the ‘prob’ variable to contain the problem data

```
prob = LpProblem('Beer Distribution Problem', LpMinimize)
```

Step 8: Creates a list of tuples containing all the possible routes for transport

```
Routes = [(w, b) for w in Warehouses for b in Bars]
```

Step 9: A dictionary called ‘Vars’ is created to contain the routes

```
vars = LpVariable.dicts('Route', (Warehouses, Bars), 0, None, LpInteger)
```

Step 10: the Objective Function is added to ‘prob’ first

```
prob += lpSum([vars[w][b]*Costs[w][b] for (w, b) in Routes]), 'Sum_of_Costs'
```

Step 11: The supply maximum constraints are added to prob for each supply node

```
for w in Warehouses:
```

```
    prob += lpSum([vars[w][b] for b in Bars])<=Supply[w], 'Sum_of_Prod_%%s'%w
```

Step 12: The demand minimum constraints are added to prob for each demand node(bar)

```
for b in Bars:
```

```
    prob+=lpSum([vars[w][b] for w in Warehouses])>=Demand[b], 'Sum_of_Prod_%%s'%b
```

Step 13: Write / Display the Linear Programming Model

```
prob.writeLP
```

Step 14 : Print Optimal Transportation Cost

```
print("\nMinimum Total Transportation Cost = $", pulp.value(prob.objective))
```

Step 15: use if statement to display all values of variables:

```
for v in prob.variables():
```

```
    print(v.name, '=', v.varValue)
```

SET :

- 1) The ICARE company has three plants located throughout a state with production capacity 50,75 and 25 gallons. Each day the firm must furnish its four retail shops R₁, R₂, R₃ and R₄ with at least 20,20,50 and 60 gallons respectively. The transportation costs(in Rs.) is given below

Company	R ₁	R ₂	R ₃	R ₄	Supply
A	3	5	7	6	50
B	2	5	8	2	75
C	3	6	9	2	25
Demand	20	20	50	60	

- 2) A beer company has two warehouses from which it distributes beer to four carefully chosen shops. At the start of every week, each bar sends an order to the brewery's head office for so many crates of beer, which is then dispatched from the appropriate warehouse to the bar. The brewery would like to have an interactive computer program which they can run week by week to tell them which warehouse should supply which bar so as to minimize the costs of the whole operation. For example, suppose that at the start of a given week the brewery has 2000 cases at warehouse A and 4500 cases at warehouse B, and that the bars required 600, 1000, 1600, 150 and 800 cases respectively. Transformation costs (dollars per crate) is given in the following table.

From Warehouse to Bar	A	B
1	3	2
2	4	3
3	6	3
4	2	1

Which warehouse should supply which bar?

- 3) The company has three plants located throughout a state with production capacity 60, 85 and 35 units of products. Each day the firm must furnish its four retail shops R_1 , R_2 , R_3 and R_4 with at least 30, 25, 55 and 65 units respectively. The transportation costs (in Rs.) are given below.

Company	R_1	R_2	R_3	R_4	Supply
A	6	5	1	6	50
B	2	5	8	2	75
C	8	7	9	2	25
Demand	25	30	50	65	

Signature: _____

Date: _____

Assignment Evaluation:

1: Late Complete

☐

2: Incomplete

☐

3: Complete

☐